基于嵌入式平台的目标检测系统研究

本科毕业论文（设计）答辩汇报

朱奕澄 | 通信与信息工程学院 | 电子信息工程

研究重点：模型迁移、RKNN 部署、实时流水线、NPU 多 context 并行与硬件优化实验

RK3588

NPU 多线程
实时目标检测

- PyTorch
- ONNX
- RKNN
- RTSP

研究背景：边缘端小目标检测需要系统级优化

无人机/小目标检测不仅是模型精度问题，也是嵌入式平台上的实时工程问题

- 远距离目标在图像中占比小，容易受天空、建筑、树木、鸟类和光照干扰。
- 嵌入式平台算力、内存带宽、视频编码和线程调度共同影响实时显示效果。
- 导师关注的核心方向是 RK3588 NPU 多线程与硬件协同优化，而不是单纯换一个更大的模型。

检测难点

小目标、背景干扰、框抖动、漏检/误检

部署难点

NPU 推理、CPU 后处理、内存拷贝、RTSP 输出

论文定位

以工程实现和实验验证为主，稳健描述边界

任务目标与完成情况

以“能跑通、可复现、可解释”为主线推进

任务线	当前状态	答辩表述
模型迁移	**已完成**	PyTorch→ONNX→RKNN，稳定主线采用 FP RKNN
实时检测	**已跑通**	固定视频和 USB 摄像头 RTSP 均可输出检测画面
NPU 多线程	**已验证**	双 context 可作为硬件并行实验重点
RGA/Zero-copy	实验路径	已有可切换实现，作为硬件优化对比而非默认路径
GPIO 报警	替代实现	无外设条件下使用软件告警叠加和日志记录

系统总体架构

从视频输入到检测结果输出采用分层、可切换的流水线设计

视频输入
文件 / USB / RTSP

预处理
OpenCV / RGA 可选

RKNN 推理
NPU context

后处理
解码、阈值、NMS

策略控制
间隔检测、ROI、跟踪

输出
视频 / CSV / RTSP

- 固定视频路径：用于可重复实验、生成带框视频、CSV、ROI JSONL 和告警事件。
- 实时 RTSP 路径：用于摄像头演示和实时观看，强调低延迟与视觉稳定性。
- 硬件优化路径：multi-context、RGA、zero-copy、profiling 均通过环境变量开关控制。

模型迁移：从训练模型到 RK3588 可执行模型

核心不是只导出模型，而是保证输出格式和后处理逻辑一致

训练权重
best.pt

中间格式
best.onnx

板端模型
best.rk3588.fp.rknn

C++ 后处理
[1,5,8400] / end2end 兼容

输出张量设计　8400 = 80×80 + 40×40 + 20×20，来自 YOLOv10 三个检测头；当前单类模型采用 5 通道输出，后处理同时兼

- 稳定主线采用 FP RKNN；INT8 已作为后续量化闭环方向，而不是当前主结论。

部署平台：RK3588 的异构计算资源

CPU、NPU、RGA、视频编解码器共同决定系统实时性

6 TOPS

NPU 算力

INT8 峰值能力，当前稳定主线为 FP RKNN

4+4

CPU 架构

Cortex-A76 + Cortex-A55 big.LITTLE

Mali-G610

图形/视频

配合硬件编解码和 RGA 减轻 CPU 负担

实时

系统目标

在低功耗板端完成检测和显示输出

- NPU 负责神经网络推理；CPU 负责线程调度、后处理、日志和策略控制。
- RGA 可承担缩放、色彩转换等图像处理任务，但是否提升端到端性能需要实测。
- 内存带宽与数据拷贝路径会影响推理前后的整体延迟。

两条运行路径：固定视频实验与实时 RTSP 演示

固定视频保证可复现，RTSP 证明真实部署能力

rk_yolo_video

输入视频文件，输出带框视频、CSV、ROI JSONL、告警 CSV

rk_yolo_live_rtsp

输入 USB 摄像头或文件，输出实时 RTSP 画面

- 论文主实验：优先使用固定视频路径，便于重复比较不同参数和硬件优化策略。
- 答辩演示：优先使用实时 RTSP 路径，可在电脑端观看带框检测画面。
- 两条路径共享 RKNN 检测器实现，减少代码分叉和实验口径不一致。

实时视频处理流水线

采集、推理、策略控制和输出解耦，避免单个慢模块拖垮系统

采集线程
读取摄像头/视频帧

任务队列
有限队列，丢弃过期帧

推理 worker
独立 RKNN context

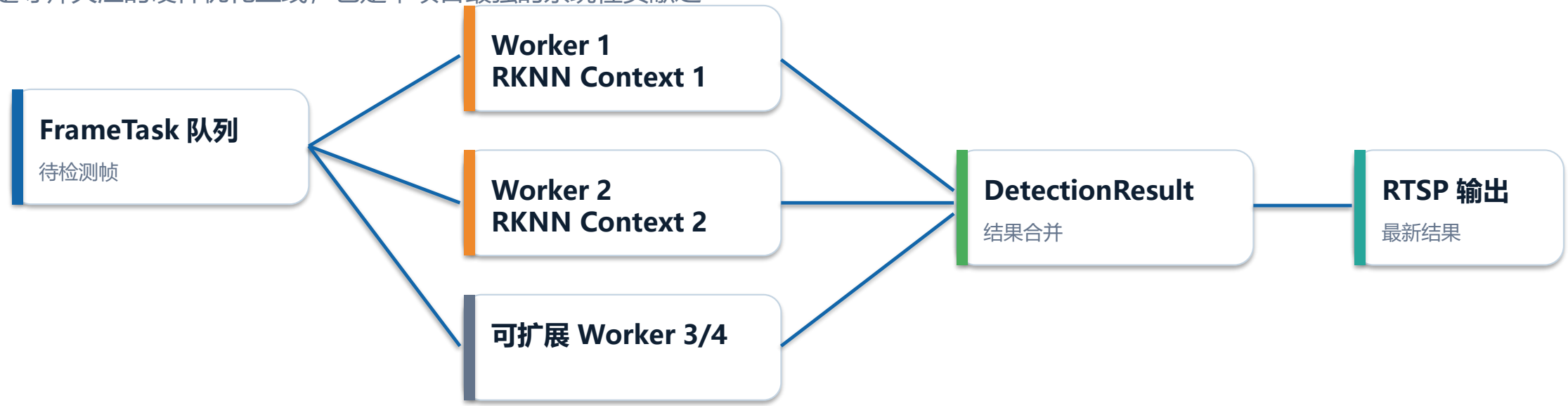
结果队列
按时间戳选择最新结果

显示/推流
画框、告警、RTSP

关键设计 用“最新帧优先 + 有限队列 + 结果复用”控制实时延迟；固定视频负责严谨对比，实时 RTSP 负责演示。

NPU 多 context 并行推理

这是导师关注的硬件优化主线，也是本项目最强的系统性贡献之一



实测结论 双 context 在每帧检测模式下提高 NPU FPS 并降低延迟；但实时观看默认仍采用策略优化路径，以获得更稳的画面体验。

实时策略：检测间隔、ROI、跟踪与平滑

目标是减少不必要推理，同时让显示框更稳定

detect_every_n

每 N 帧执行一次完整 NPU 检测；中间帧复用或跟踪结果。

动态 ROI

围绕历史目标区域裁剪并周期性全帧刷新，兼顾速度和漏检风险。

轻量跟踪

motion 线性位移预测；optflow 使用 Lucas-Kanade 稀疏光流。

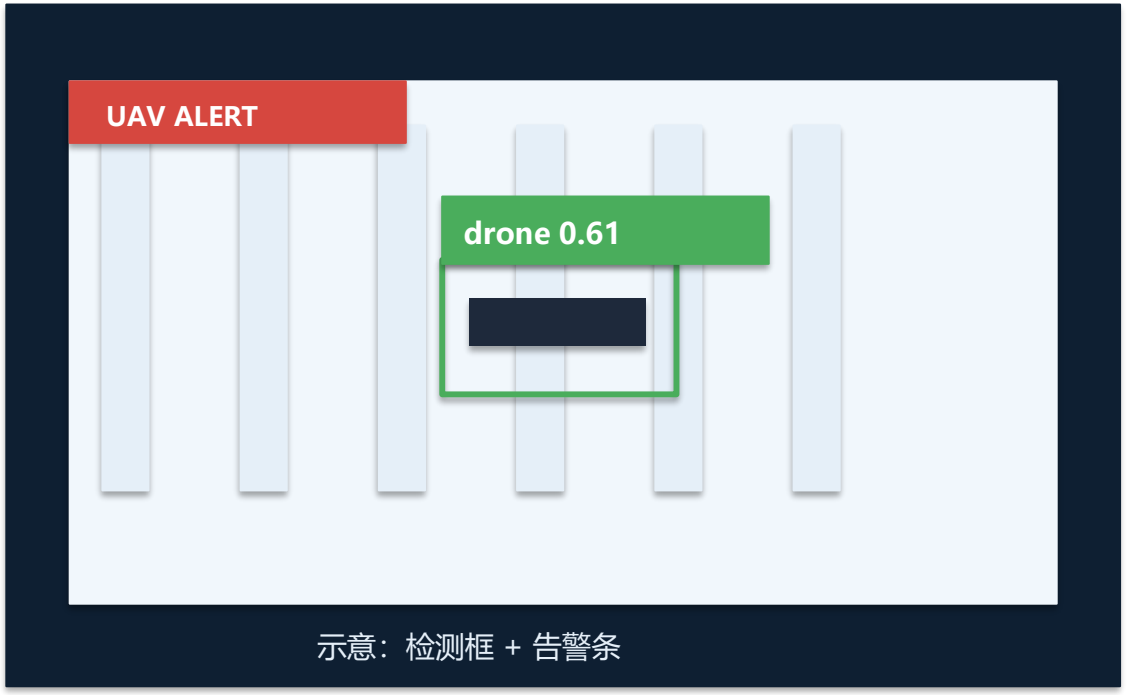
框平滑

对检测框坐标做加权融合，减少左右抖动。

推荐演示：detect_every_n=2，motion tracking，box smoothing on，dynamic ROI on。

检测效果与可视化输出

固定视频和实时 RTSP 均支持检测框、类别置信度和告警叠加



示意：检测框 + 告警条

- 离线视频：输出带框 MP4、逐帧 CSV、ROI JSONL 和 alarm_events.csv。
- 实时演示：电脑端可通过 RTSP 地址查看检测画面。
- 无继电器/蜂鸣器条件下，软件告警提供更直观、可记录的替代反馈。

实验对比：优化不是“硬件越多越好”

最终推荐基于端到端效果，而不是单个算子是否硬件加速

硬件基线

单 context 每帧检测

NPU FPS≈8.59
延迟≈268.89 ms

并行推理证据

双 context 每帧检测

NPU FPS≈10.64
延迟≈173.58 ms

演示默认

策略优化实时显示

detect_every_n=2
跟踪+平滑+ROI

优化实验

RGA/zero-copy

部分阶段有效
需端到端判断

论文结论口径 多 context 证明 NPU 并行能力；实时演示采用更稳的策略优化配置；RGA 与 zero-copy 作为可切换实验路径记录正负

性能剖析与硬件优化路径

通过环境变量开关做可重复实验，避免把探索路径写成默认结论

RK_YOLO_PROFILE

prepare / input / run / output / decode / render

RK_YOLO_ZERO_COPY_INPUT

rknn_create_mem + rknn_set_io_mem

RK_YOLO_PREPROCESS=rga

RGA resize / cvt_resize / letterbox

当前判断

- zero-copy 输入上传阶段变快，但整体 `rknn_run` 上升，未作为默认。
- RGA 在高分辨率输入和部分预处理路径上有收益，但仍需按端到端延迟判断。
- INT8 量化需要校准集、精度对比和板端闭环，当前作为后续优化。

目标调整与不足说明

答辩时建议正面说明：哪些已完成，哪些保留为后续工作

INT8 量化	已做离线准备，缺少完整精度-速度闭环	后续补充校准集和板端对比
RGA 主路径	已有多条实验路径，默认仍选择稳定 OpenCV/FP 路线	继续优化 MPP/RGA 数据通路
GPIO 报警	无继电器/蜂鸣器硬件	软件告警叠加 + alarm CSV 作为可视替代
真实飞行场景	受场地与无人机条件限制	公开视频 + 教室小目标演示补充验证

总结

本课题完成了从模型到 RK3588 板端实时检测系统的工程闭环，并围绕 NPU 多线程与硬件优化做

系统闭环

训练/转换/部署/固定视频/RTSP 显示

优化闭环

detect_every_n、tracking、ROI、multi-context、RGA

论文闭环

实验记录、代码映射、格式合规、附录材料

答辩主线：我不是提出一个全新的检测网络，而是在 RK3588 上完成了可运行、可测量、可解释的嵌入式目标检测系统。

谢谢各位老师